DARWIN Voice Agent Architecture and Implementation Plan

An evidence based systems design for reliable wake word detection barge in and natural conversation on macOS

DARWIN already has most of the recognizable voice-agent chain: continuous microphone capture, wake-word detection, voice activity detection, speech recognition, agent orchestration, streaming speech synthesis, and media ducking. The unresolved failure is not primarily a language-model problem or a threshold problem. It is an audio-front-end problem: during loudspeaker playback, the microphone receives a mixture in which foreground music can mask the user before the wake model sees the signal.

The recommended strategy is to stop full wake-model retraining until the capture path is measured and improved. First establish an Apple-native voice-processing prototype that provides acoustic echo cancellation, noise suppression, and automatic gain control. Route the cleaned capture stream into the existing Python pipeline. Then evaluate the unchanged v1 wake model on synchronized raw and processed audio. Only after that evidence exists should DARWIN decide whether a new wake model, a personalized second-stage verifier, or additional speech enhancement is necessary.

Executive decision

Build DARWIN as a layered realtime audio system, not as a sequence of independent ML models. The decisive architectural boundary is the acoustic front end. It must own input and output timing, maintain a playback reference or use Apple voice processing, and expose both cleaned audio and objective telemetry. Wake detection, VAD, ASR, speaker identity, and interruption control should consume that cleaned stream through explicit state machines.

Decision	Recommendation	Reason
First engineering move	Native macOS audio-front-end spike using AVAudioEngine voice processing or AUVoiceIO	Tests the most plausible fix before more training
Active wake model	Keep darwin_v1.onnx at the current validated operating point	media_v2 has 0.99 percent media recall and is not viable
Media handling	AEC first; modest ducking as a fallback and during response playback	Ducking after speech begins cannot recover the wake word
Conversation path	Keep the chained pipeline initially; make every stage streaming and cancellable	It is inspectable, testable, and preserves current integrations
Speaker identity	Use as personalization or risk signal only	Replay and synthetic speech defeat naïve

Decision	Recommendation	Reason
		voice biometrics
Release criterion	Condition-specific recall plus false accepts per hour and latency percentiles	Average accuracy conceals deployment failures

How production voice agents function

A production voice assistant is usually two coupled systems. The realtime media plane captures, cleans, buffers, detects, transcribes, and plays audio under tight timing constraints. The conversation plane interprets turns, invokes tools, manages safety and dialogue state, and generates speech. Treating these as one synchronous request chain creates latency, self-transcription, dropped opening phonemes, and fragile interruptions.

The realtime media plane

1. Capture fixed-duration frames from a stable device clock. Preserve timestamps, detect dropped frames, and retain a short pre-roll ring buffer.
2. Apply acoustic echo cancellation when loudspeaker output is present, using a time-aligned render reference or a platform voice-processing path. Then apply conservative noise suppression and automatic gain control.
3. Run speech activity and wake-word inference continuously. Smooth scores over time, require a trigger pattern, and impose a refractory period after activation.
4. After activation, stream audio into ASR while endpointing continues. Partial transcripts can prepare the conversation engine, but only stable/final text should authorize tools.
5. Stream the response and TTS audio as soon as useful content is ready. Maintain an exact record of what was actually played so cancellation or interruption does not leave unheard text in conversation state.
6. Continue capture during playback. Use cleaned audio plus interruption logic to distinguish a real barge-in from coughs, backchannels, music, and residual echo.

Apple exposes voice processing through AVAudioEngine and AUVoiceIO. Apple describes the path as device-tuned echo cancellation, noise suppression, and automatic gain control; enabling AVAudioEngine voice processing places both input and output nodes in voice-processing mode. Voice processing must be configured while the engine is stopped and operates when rendering to a real audio device.[12]

WebRTC AEC3 illustrates why this layer needs both render and capture signals: its API separately analyzes render audio, analyzes capture audio, estimates delay, handles saturation, and processes capture frames. AEC is an adaptive system, not a static filter. Double-talk, device delay, clock drift, nonlinear speaker distortion, clipping, and changes in the acoustic path are normal hard cases.[34]

The conversation plane

The conversation layer should be event-driven: speech started, partial transcript, turn committed, response started, tool pending, audio playing, interruption suspected, interruption confirmed, and response completed. A finite-state controller must be authoritative even if individual services run concurrently. This prevents two recorders from competing for the device and makes cancellation deterministic.

Modern realtime APIs expose the same primitives explicitly. Server VAD typically includes prefix padding, a speech threshold, a silence duration, and automatic response interruption. Semantic turn detection goes beyond silence by estimating whether the utterance is complete. These controls reflect an unavoidable tradeoff: short endpoint delays feel fast but cut off thoughtful pauses; long delays preserve speech but feel unresponsive.[56]

Two viable conversational architectures

Architecture	Strengths	Weaknesses	DARWIN fit
Chained STT to LLM to TTS	Transcripts are inspectable; tools and policies remain explicit; providers can be swapped	Every boundary adds latency; prosody and nonverbal cues are lost	Recommended near term because DARWIN already uses it
Native speech to speech	Natural timing, affect, interruption behavior, and potentially lower orchestration latency	Harder to inspect and reproduce; provider dependence; audio privacy and cost	Evaluate later behind the same media-plane interface

The pipeline should remain modular even if a speech-to-speech model is adopted. A realtime model does not replace local wake detection, acoustic echo cancellation, capture reliability, permission handling, or action confirmation.

The central acoustic problem

Echo is not the same as external interference

AEC can remove playback only when the system has a correlated reference for that playback. DARWIN's own TTS is easy to reference because DARWIN generates it. Spotify or Apple Music playback belongs to another process, so the implementation must verify whether Apple voice processing cancels system output on the target Mac and route. If it does not, DARWIN needs a system-output capture or a virtual audio route to provide the reference, which increases permissions, routing complexity, and failure modes.

Even perfect echo cancellation cannot remove uncorrelated sound from an external television, phone, or person. Those cases require spatial advantages such as microphone arrays and beamforming, speech enhancement, closer microphones, or lower interference volume. A single laptop microphone has fundamental limits when music vocals are louder than the user at the microphone.

Masking explains the failed threshold strategy

The observed v1 misses near 0.001 while music is playing show that the wake phrase often disappears from the learned representation. Lowering the decision threshold cannot reconstruct missing evidence. It only admits more non-wake events. The failed media_v2 experiment reinforces this: its synthetic media-positive and media-negative score distributions overlap heavily, while the aggregate training loss falls because clean original examples remain easier and numerous.

Research on keyword spotting under playback interference explicitly assumes residual echo after AEC and augments training with music and television at varied signal-to-interference ratios. That is evidence for augmentation after an AEC baseline, not evidence that augmentation can substitute for AEC.[7]

Wake words are detection systems not clip classifiers

A streaming wake system produces frame-level scores and then applies temporal decision logic. Production designs commonly add posterior smoothing, peak detection, trigger-level requirements, debounce or refractory windows, and a second-stage verifier. Apple reports two networks for Hey Siri: an initial detector and a secondary checker. Amazon has published two-stage wake-word systems and richer background models to reduce both false rejects and false alarms.[8][9][10]

openWakeWord processes 16 kHz PCM in multiples of 80 ms and uses a streaming sliding feature window. Its documentation recommends environment-specific threshold testing, optional VAD gating to reduce non-speech false positives, and a custom verifier as a second-stage filter for known voices. It also recommends large synthetic datasets and long false-positive validation data rather than a handful of clips.[11][12]

The correct metrics

Metric	Meaning	How DARWIN should report it
False reject rate	Fraction of genuine wake attempts missed	Per condition, distance, level, route, and speaking style
False accepts per hour	Spurious activations normalized by exposure time	Ordinary speech, music, television, silence, and mixed ambient streams
Detection latency	Time from the acoustic end of the wake phrase to trigger	Median, p90, p95, and p99
Endpoint latency	Time from the user's last speech to committed turn	Split VAD and semantic endpoint contribution
First response audio	Time from committed turn to audible response	Separate ASR, model, TTS, network, and playback components
Interruption precision and recall	Whether true barges stop playback without false interruptions	Score by playback type and near-end phrase

Wake-word accuracy must be represented as a tradeoff curve rather than one accuracy number. Apple defines false accepts per hour and false rejects per attempted activation. Amazon production research evaluates DET curves and reports that production-grade systems use hundreds of hours of far-field data and hundreds of hours of evaluation audio across acoustic conditions.[8][13]

Why the media v2 experiment failed

The run is useful because it rules out a simplistic answer. The training loss decreased from 1.17 to 0.37, yet media recall at the selected threshold was 0.99 percent. Inspection of the cached scores shows that synthetic media-positive and media-negative distributions overlap in both training and validation. The best checkpoint was selected by aggregate training loss, not by validation utility or a constrained false-accept operating point.

Observed issue	Consequence	Required correction
Original clean examples dominate aggregate optimization	Loss can improve while media discrimination remains poor	Track loss and DET metrics separately for every condition
One short recording session	Song, room, distance, and device leakage cross-contaminate train and validation	Split by complete session and playback source
Synthetic positive and negative scores overlap	More epochs reinforce a representation that lacks separability	Audit mixtures and embeddings before retraining
Best model chosen by training loss	No protection against generalization collapse	Select by validation FRR at a fixed false-accept budget
Headline media metric dominated by 400 synthetic cases	Three real held-out positives are obscured	Report real and synthetic strata independently

Do not resume this recipe, increase epochs, or lower the threshold. Preserve the artifacts for diagnosis and establish a clean baseline after front-end processing.

Recommended DARWIN architecture

The following architecture preserves DARWIN's working application logic while replacing the weak boundary. The native helper should be a small, independently testable macOS component rather than a rewrite of the application.

Layer	Responsibility	Interface
Native audio service	Own AVAudioEngine or AUVoiceIO, device changes, timestamps, AEC, NS, AGC, playback reference, route telemetry	Framed PCM stream plus events over local IPC
Audio router	Ring buffers, resampling, raw and processed taps, fan-out, backpressure	One producer and explicit named consumers
Wake service	Streaming v1 inference, smoothing, trigger policy, refractory period, optional second-stage verifier	WakeCandidate and WakeAccepted events
Turn manager	Pre-roll, VAD, semantic endpointing, follow-up policy, interruption confirmation	State transitions with monotonic timestamps
ASR service	Streaming partial and final hypotheses with confidence and timing	TranscriptDelta and TranscriptFinal
Conversation runtime	Context, tools, safety, confirmations, cancellation, concise response planning	Structured user and assistant turns
TTS and playback	Sentence or token streaming, cancellable playback, exact played-audio ledger	PCM render stream and played range events
Telemetry and replay	Stage latency, audio quality, scores, failures, privacy-controlled diagnostic clips	Structured local event log

State machine

Idle listens locally for the wake word. Candidate temporarily retains pre-roll and may run a verifier. Capturing streams the command to ASR. Thinking keeps capture active for possible cancellation. Speaking plays TTS while processed capture continues for barge-in. Follow-up accepts speech without another wake word for a bounded, dialogue-aware interval. Suspended is used during permission loss, route changes, or unrecoverable audio faults.

Every transition must be cancel-safe. A confirmed interruption stops playback, cancels queued TTS, truncates the assistant turn to the audio actually heard, clears stale capture buffers, and promotes the interrupting speech into the next user turn. A suspected interruption should attenuate playback briefly while evidence accumulates; it should not destroy the response immediately.[56]

A phased implementation plan

Phase 0 Preserve and instrument

- Freeze darwin_v1.onnx as the reference and explicitly blacklist media_v2 from production configuration.
- Add monotonic timestamps for capture callback, wake score, wake trigger, speech start and stop, ASR partial and final, model first token, TTS first byte, playback first frame, interruption, and playback completion.

- Store per-frame overflow counts, queue depth, selected input and output devices, sample-rate conversion, clipping rate, RMS, and media state.
- Create privacy modes: metrics only by default; diagnostic audio only with an explicit local opt-in and retention limit.

Phase 1 Native audio front end

- Build a Swift command-line prototype that enables AVAudioEngine voice processing before starting the engine and emits 16 kHz mono PCM with timestamps.
- Record synchronized raw and processed streams where APIs permit, plus the known DARWIN TTS render stream.
- Test four routes: built-in microphone plus built-in speakers, wired or USB headset, Bluetooth route, and external microphone plus speakers.
- Test three playback sources separately: DARWIN TTS, Spotify or Apple Music, and an external phone or television. This distinguishes cancellable device output from uncorrelated room noise.
- Measure wake recall with unchanged v1. The go decision is based on improvement at the same false-accept budget, not subjective cleanliness.

Apple recommends voice-processing APIs for two-way voice applications and states that the processing is tuned by acoustic engineers for each Apple product and audio-device type. That makes the platform path the highest-value first experiment on the target hardware.[2]

Phase 2 Deterministic evaluation harness

- Define immutable manifests for clean positive, processed media positive, raw media positive, hard-negative speech, music-only, television, ambient, and assistant-playback corpora.
- Split by recording session, song or program, room, route, and day. No clip or background source may appear on both sides of a split.
- Run the actual streaming detector, including its sliding state, smoothing, trigger policy, resets, and refractory period. Clip-level classifier evaluation is insufficient.
- Generate score histograms, ROC, precision-recall and DET curves; report confidence intervals when sample counts are small.
- Maintain a long negative soak corpus. Production research evaluates hundreds of hours; DARWIN should grow from tens of hours to hundreds before claiming production reliability.

Phase 3 Wake decision improvements

- Tune smoothing, trigger count, hysteresis, and refractory behavior on frozen validation data.
- Use VAD as a false-positive gate only after verifying it does not reject heavily masked genuine wake speech.
- Add a speaker-personalized verifier only as a second stage after a permissive general wake candidate. Do not require it to rescue audio that the first stage never detects.
- If retraining remains necessary, train with post-AEC residuals captured from real devices, varied playback material, realistic SIR distributions, room impulse responses, nonlinear distortion, and strict session separation.
- Reject any candidate automatically if clean recall, media recall, false accepts per hour, or detection latency crosses a release limit.

Phase 4 Natural conversation

- Move Faster Whisper from record-then-transcribe toward true streaming or chunked partial hypotheses.
- Combine acoustic VAD with transcript stability and semantic end-of-turn prediction; keep a configurable maximum wait.
- Begin response planning on stable partial text, but do not execute consequential tools until the user turn is committed.
- Make TTS and playback cancellable at frame boundaries and record exactly what audio was heard.

- Use an interruption classifier or confirmation window to reject backchannels and residual playback without making barge-in feel sluggish.

Phase 5 Security reliability and onboarding

- Separate attention from authorization. Wake words and voiceprints may personalize behavior but cannot authorize sensitive actions.
- Require explicit confirmation, operating-system authentication, or another strong factor for payments, messages, destructive actions, credentials, and privacy-sensitive access.
- Implement first-run microphone, accessibility, automation, media, and network disclosure flows with actionable recovery states.
- Provide offline and networked modes with clear disclosure of which audio or text leaves the device.

Voice biometrics are vulnerable to replay, synthesis, and voice conversion. ASVspoof exists specifically to measure these attacks, and NIST notes that replayed recordings are a common bypass. DARWIN's current ECAPA comparison therefore remains an attention signal, not a security boundary.[14][15][16]

Acceptance gates

The exact targets should be finalized after baseline measurement, but the project needs explicit provisional gates now. These are engineering recommendations, not published industry standards.

Gate	Prototype target	Release target
Clean wake recall	At least 95 percent	At least 98 percent across held-out sessions
Media wake recall	At least 80 percent at normal listening level	At least 90 percent by defined SIR and distance bands
False accepts	No more than 0.5 per hour	No more than 0.1 per hour over at least 100 negative hours
Wake detection p95	No more than 500 ms after phrase end	No more than 350 ms
Turn endpoint p95	No more than 1.0 s for short commands	Dynamic endpointing with no material truncation regression
First audible response p95	No more than 1.8 s after turn commit	No more than 1.2 s on a healthy network
Barge-in	Stops within 500 ms with controlled false interruptions	At least 95 percent recall and 98 percent precision in test corpus
Audio integrity	No dropped frames in a 2 hour soak	No unrecovered audio failure in repeated overnight soaks

Experiments that resolve the architecture

7. Apple voice-processing A B test. Replay the same positive and negative corpus through the target Mac speakers while simultaneously recording. Compare raw PortAudio capture, AVAudioEngine voice-processed capture, and headset capture with v1 held constant.
8. Other-application playback test. Run Spotify through the system output and determine empirically whether the chosen Apple path receives a useful playback reference. Do not infer this from API naming.
9. SIR sweep. Measure at controlled voice-to-interference ratios from easy to hard, using multiple distances and songs. Find the physical failure boundary rather than one aggregate recall.

10. External-interference control. Repeat with music from a separate phone. Any difference from system playback estimates what AEC contributes and what remains for speech enhancement or hardware.
11. Route-change and convergence test. Start playback immediately, change volume, move the laptop, and switch devices. Measure the initial convergence interval and recovery after echo-path changes.
12. End-to-end barge-in test. Play DARWIN TTS and speak over it. Confirm capture quality, interruption timing, cancellation of queued speech, and correct conversation-state truncation.

Risks and boundaries

Risk	Reality	Mitigation
System playback is unavailable as an AEC reference	Platform APIs may behave differently across routes and other applications	Empirical native prototype; fallback to ducking, headphones, or explicit system-audio routing
Single microphone physical limit	Loud uncorrelated music can mask near-end speech	Set supported operating envelope; add external microphone or array hardware if needed
Aggressive suppression damages wake phonemes	Enhancement can improve perceived audio but reduce detector evidence	Optimize against downstream wake and ASR metrics, not perceptual quality alone
Bluetooth route changes quality and latency	Profiles and sample rates vary	Route-specific baselines and graceful capability reporting
Cloud dependency	ASR, model, or TTS latency and availability fluctuate	Timeouts, cancellation, local fallback, stage telemetry, concise responses
Data leakage	Diagnostic audio and remote speech services are sensitive	Local-first metrics, explicit opt-in, retention and provider disclosure

What success looks like

A successful DARWIN implementation will not be defined by one impressive demo. It will show stable audio capture, predictable behavior across routes, condition-specific wake performance, low false accepts over long exposure, fast endpointing without clipping, and deterministic interruption. When a condition exceeds the physical capability of the hardware, DARWIN should degrade deliberately: request a repeat, lower or pause controllable media after a candidate signal, suggest a closer microphone, or require push-to-talk.

The immediate decision is therefore narrow and testable: build the native voice-processing spike and the evaluation harness before any further media wake-word training. If v1 recall improves materially on processed audio, invest in the native front end. If it does not, inspect synchronized raw/processed mixtures and only then evaluate a new representation or hardware path. This sequence turns the project from model guessing into systems engineering.

Sources

1. Apple. What's New in AVAudioEngine. 2019. https://developer.apple.com/videos/play/wwdc2019/510/
2. Apple. What's New in Voice Processing. 2023. https://developer.apple.com/videos/play/wwdc2023/10 235/
3. WebRTC. EchoCanceller3 implementation. accessed 2026. https://webrtc.googlesource.com/src/+/refs/heads/main/modules/audio_processing/aec3/echo_canceller3.cc
4. Haubner et al. End To End Deep Learning Based Adaptation Control for Linear Acoustic Echo Cancellation. 2023. https://arxiv.org/abs/2306.02450
5. OpenAI. Realtime API reference. accessed 2026. https://platform.openai.com/docs/api-reference/realtime
6. LiveKit. Turn Taking Tuning. accessed 2026. https://docs.livekit.io/agents/logic/turns/tuning/
7. Braun et al. Data Augmentation for Robust Keyword Spotting under Playback Interference. 2018. https://arxiv.org/abs/1808.00563
8. Apple Machine Learning Research. Hey Siri An On Device DNN Powered Voice Trigger. 2017. https://machinelearning.apple.com/research/hey-siri
9. Amazon Science. Monophone Based Background Modeling for Two Stage On Device Wake Word Detection. 2018. https://www.amazon.science/publications/monophone-based-background-modeling-for-two-stage-on-device-wake-word-detection
10. Amazon Science. Alexa Scientists Present Two New Techniques That Improve Wake Word Performance. 2018. https://www.amazon.science/blog/alexa-scientists-present-two-new-techniques-that-improve-wake-word-performance
11. David Scripka. openWakeWord README. accessed 2026. https://github.com/dscripka/openWakeWord/blob/main/README.md
12. David Scripka. openWakeWord Custom Model Configuration. accessed 2026. https://github.com/dscripka/openWakeWord/blob/main/examples/custom_model.yml
13. Li et al. Small Footprint Keyword Spotting with Multi Task Learning and Transfer Learning. 2020. https://assets.amazon.science/7c/b2/5e3e6a164 920bfc167fb5586d3f2/scipub-1260.pdf
14. ASVspoof. Automatic Speaker Verification Spoofing and Countermeasures Challenge. accessed 2026. https://www.asvspoof.org/
15. NIST. Mobile Threat Catalogue Biometrics. 2015. https://nvlpubs.nist.gov/nistpubs/ir/2015/NIST.IR.8014.pdf
16. Greenberg et al. Two Decades of Speaker Recognition Evaluation at NIST. 2020. https://www.nist.gov/publications/two-decades-speaker-recognition-evaluation-national-institute-standards-and-technology

Project evidence

DARWIN Voice and Speech Handoff, September 10 2026; DARWIN repository implementation and darwin_media_v2 evaluation output. These internal materials support statements about the current pipeline, model settings, experiments, and measured media_v2 results.